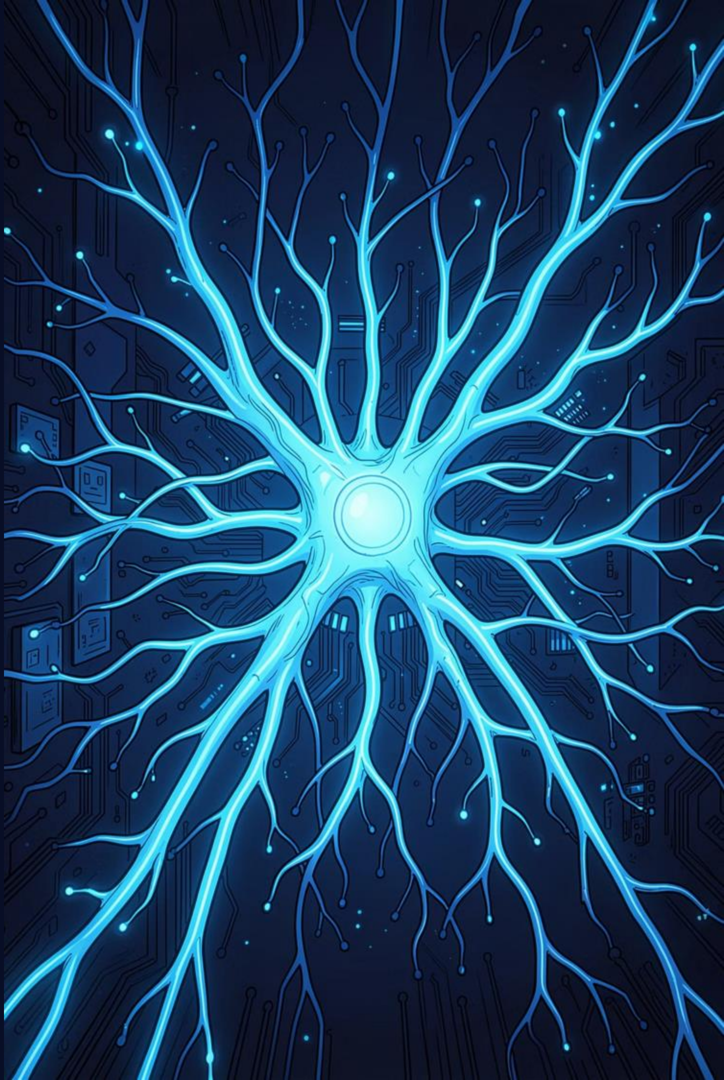
Large Language Models: How They Work

A technical walkthrough of the transformer architecture – from tokenization to text generation – with real examples at every step.

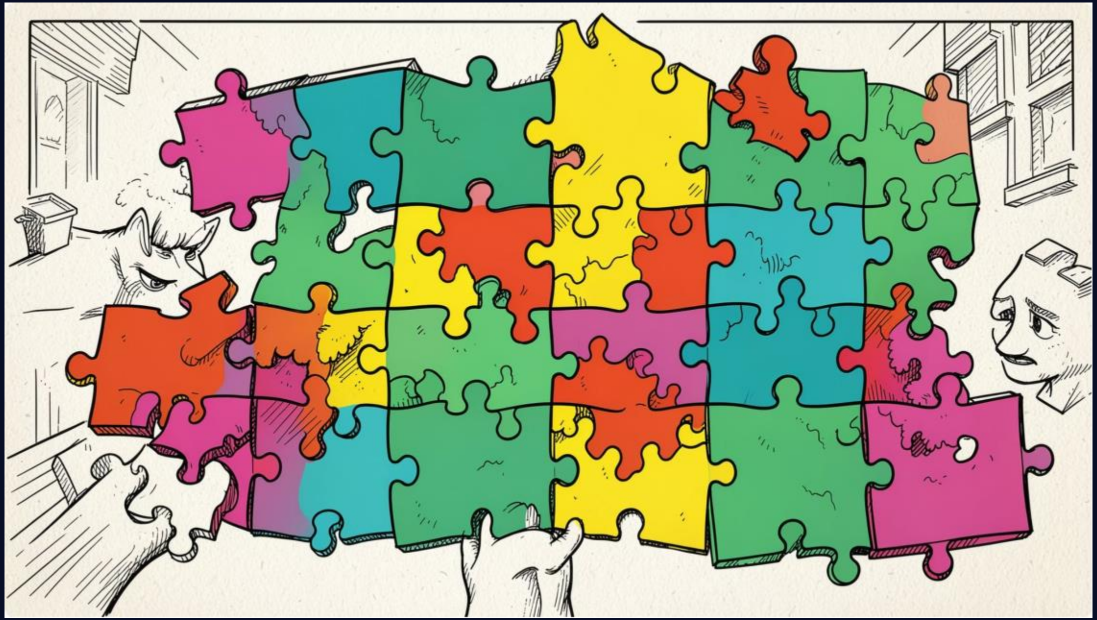
Presented by Narin Bilal



What Is an LLM?

A large language model is a neural network trained to predict the next token in a sequence – at massive scale. Given "**The capital of France is ____**", it outputs "**Paris.**"

Tokenization: Text to Subword Units



LLMs don't read raw text – they read **tokens**, subword units from a shared vocabulary.

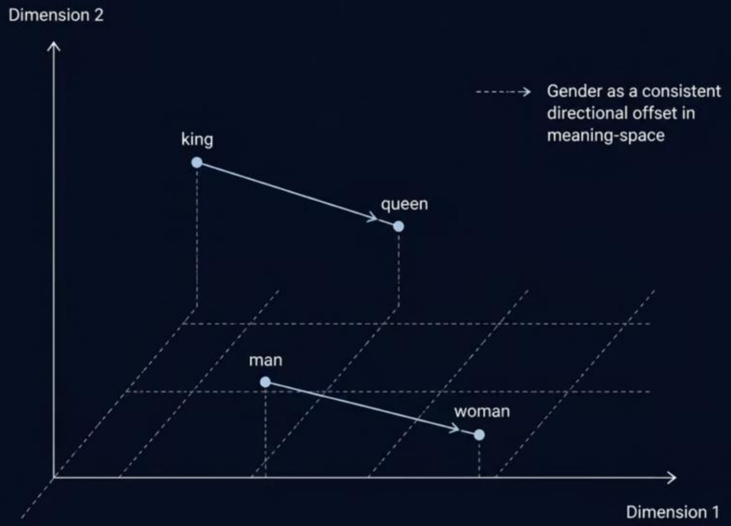
"playing" → **"play"** + **"ing"**

Rare words decompose into known pieces, keeping vocabulary compact while generalizing to unseen terms.

Embeddings: Meaning as Vectors

Each token maps to a high-dimensional vector encoding semantic meaning. Similar words land **close together** in vector space.

king - man + woman ≈ queen



The Transformer: Born in 2017

Vaswani et al.'s paper "**Attention Is All You Need**" proposed replacing recurrence entirely with **self-attention** – enabling full parallelization and dramatically faster training at scale.

1. **2017**

   Original Transformer paper – encoder-decoder for translation

2. **2018**

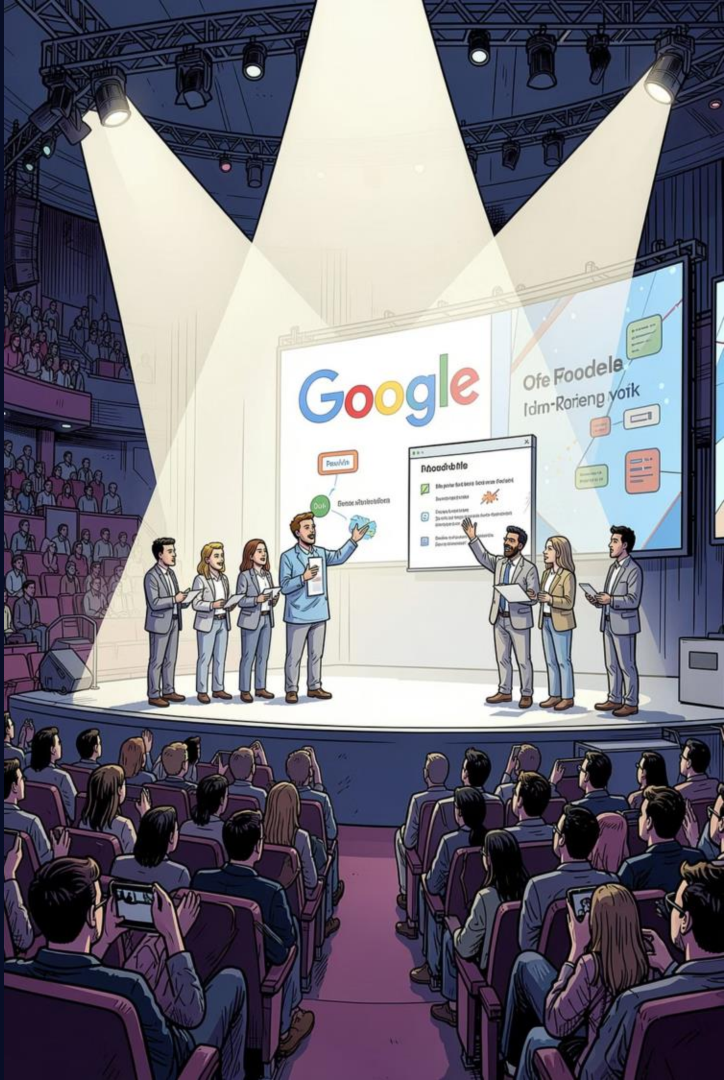
   BERT (encoder-only) and GPT (decoder-only) adapt the architecture

3. **2019–22**

   T5, GPT-3, and BART scale transformers to billions of parameters

4. **2023+**

   GPT-4, Claude, LLaMA, Gemini define the modern LLM era



Inside a Transformer Block

Input

Token embeddings + positional encoding

Feed-Forward

Position-wise MLP applied per token

Self-Attention

Multi-head: each token attends to all

Output

Residuals + layer norm; blocks stacked

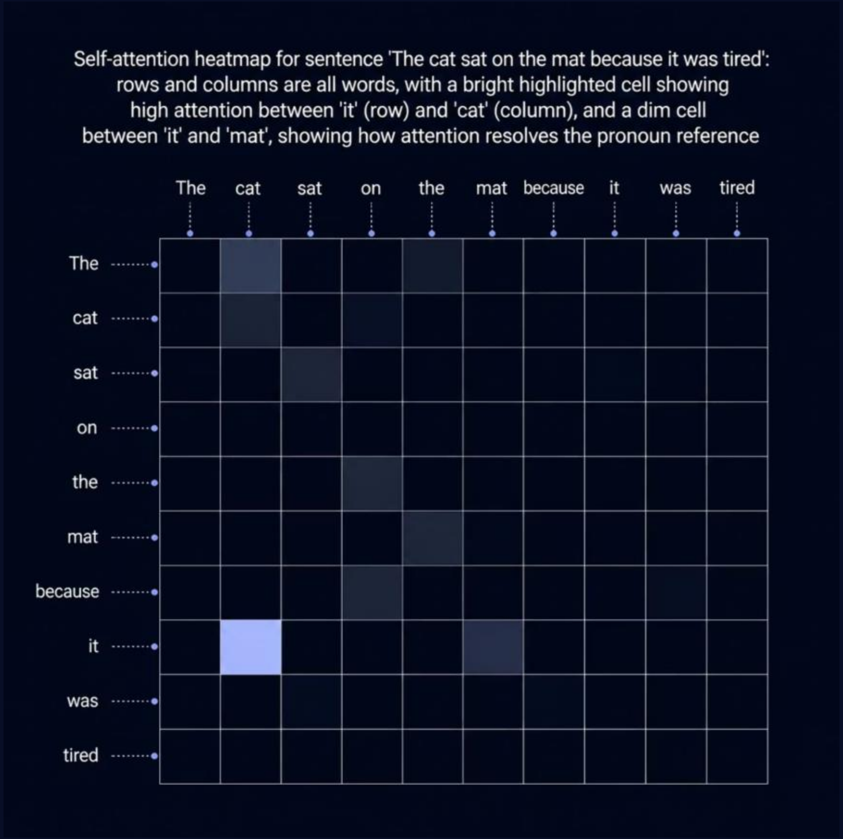
Each block has two sub-layers – self-attention and a position-wise feed-forward network – both wrapped with **residual connections** and **layer normalization**. Stacking many blocks builds the full model depth.

Self-Attention: Every Word Sees Every Other

Every token attends to all others simultaneously learning what is relevant.

> "The cat sat on the mat because **it** was tired."

Self-attention assigns a **high score** between *"it"* and *"cat"* resolving the pronoun correctly.



Query, Key & Value

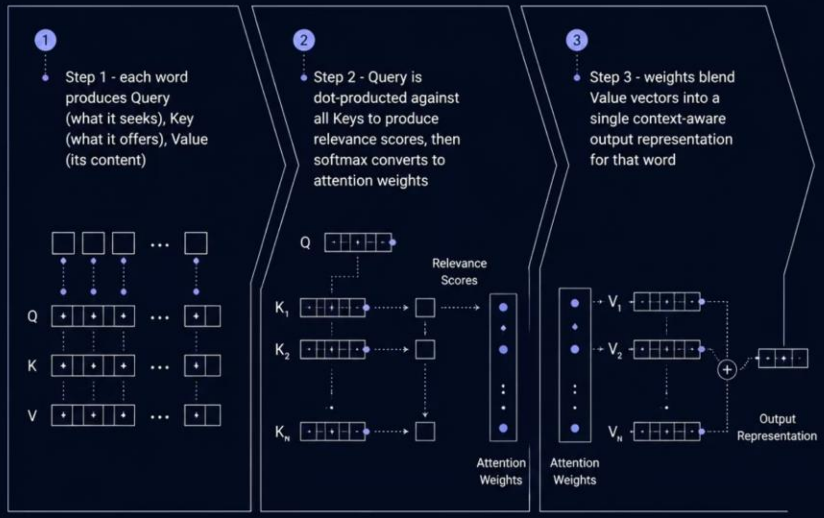
Each token produces three vectors:

- **Query (Q):** what this token is looking for
- **Key (K):** what it offers others
- **Value (V):** its actual content

Q is compared against all Keys; softmax turns scores into weights that blend Values Into a context-aware output.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

The $\sqrt{d_k}$ scaling term keeps gradients stable during training.



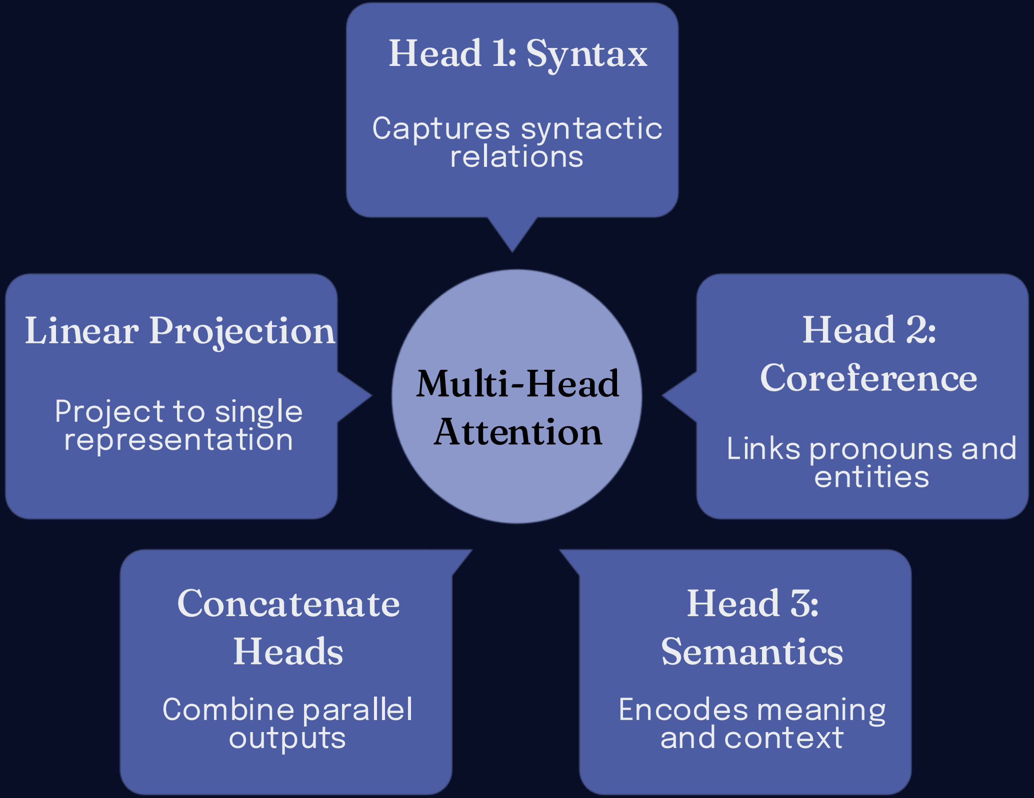
Multi-Head Attention & Positional Encoding

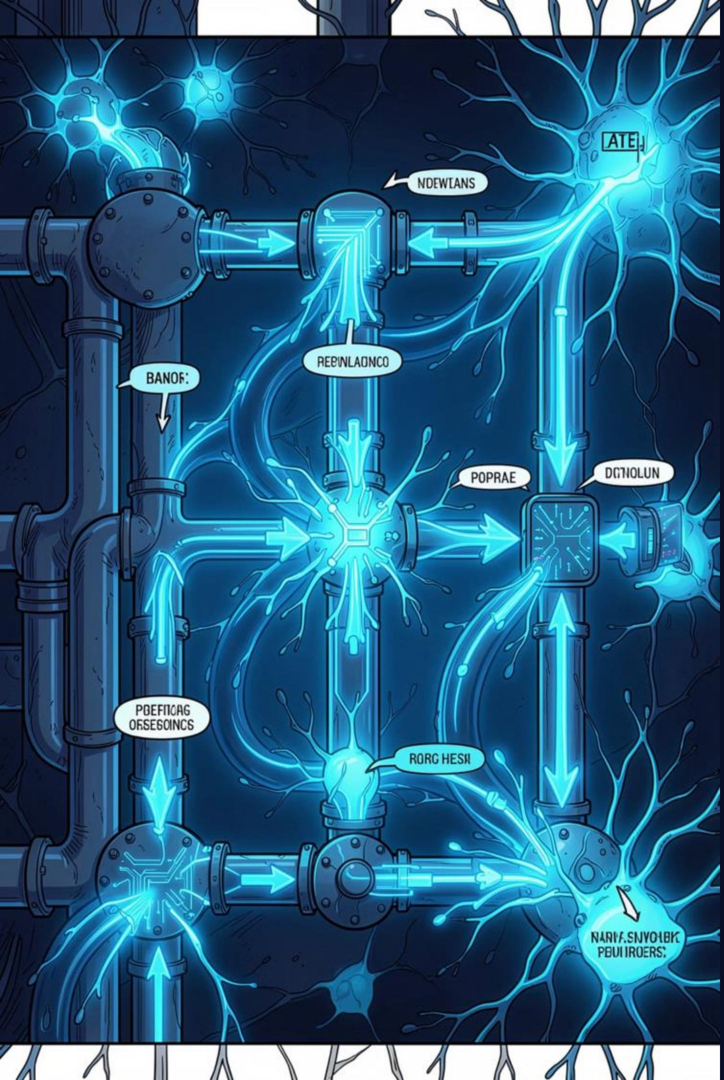
Multi-Head Attention

The transformer runs **multiple attention heads in parallel**, each learning different relationships – syntax, coreference, semantics. Results are concatenated and projected, giving a richer representation.

Positional Encoding

Since attention processes all tokens simultaneously, a **position vector** is added to each embedding to preserve word order. Without it, *"dog bites man"* and *"man bites dog"* would be indistinguishable.





Architecture Variants, Training & Full Pipeline

Encoder-Only (BERT)

Bidirectional attention for understanding and classification.

Decoder-Only (GPT, Claude, LLaMA)

Masked attention for autoregressive text generation.

Encoder-Decoder (T5)

Cross-attention bridges encoding and generation for translation.

Training: Pretraining → Fine-tuning → RLHF aligns outputs to be helpful and safe.

Full Pipeline: Text → Tokens → Embeddings + Positional Encoding → Transformer Blocks (Multi-Head Attention + Feed-Forward) → Next-Token Prediction → Repeat